

LAB 3: Logistic Regression

Mục tiêu

- Triển khai mô hình Hồi quy Logistic (Logistic Regression) bằng thư viện Numpy để dự đoán kết quả của bài toán phân loại nhị phân.
- Hiểu vai trò và cơ chế hoạt động của hàm Sigmoid.
- Hiểu về các chỉ số để đánh giá mô hình phân loại: Accuracy, Precision, Recall, F1-score.

Giới thiệu

Hồi quy Logistic là thuật toán học máy có giám sát dùng cho bài toán phân loại nhị phân. Dù mang tên "hồi quy", đây thực chất là phương pháp phân loại, không dự đoán giá trị liên tục. Thuật toán hoạt động bằng cách áp dụng hàm Sigmoid lên tổ hợp tuyến tính của các đặc trưng đầu vào, cho ra xác suất trong khoảng $[0, 1]$, rồi so sánh với một ngưỡng quyết định để gán nhãn phân loại.

Giả sử ta có tập dữ liệu huấn luyện với n đặc trưng $X = [x_1, x_2, \dots, x_n]$ và nhãn $y \in \{0, 1\}$. Mô hình hồi quy Logistic thực hiện các bước sau:

1. Tính tổ hợp tuyến tính của các đặc trưng:

$$z = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n = \mathbf{w}^T \mathbf{x} \quad (1)$$

trong đó:

- $\mathbf{w} = [w_0, w_1, \dots, w_n]$: vector trọng số (bao gồm bias w_0).
- $\mathbf{x} = [1, x_1, \dots, x_n]$: vector đặc trưng (đã thêm 1 để tính bias).

Ví dụ: nếu $x_1 = \text{tuổi}$, $x_2 = \text{huyết áp}$, thì $(z = w_0 + w_1 \cdot \text{tuổi} + w_2 \cdot \text{huyết áp})$ biểu diễn mức độ ảnh hưởng của các đặc trưng đến kết quả.

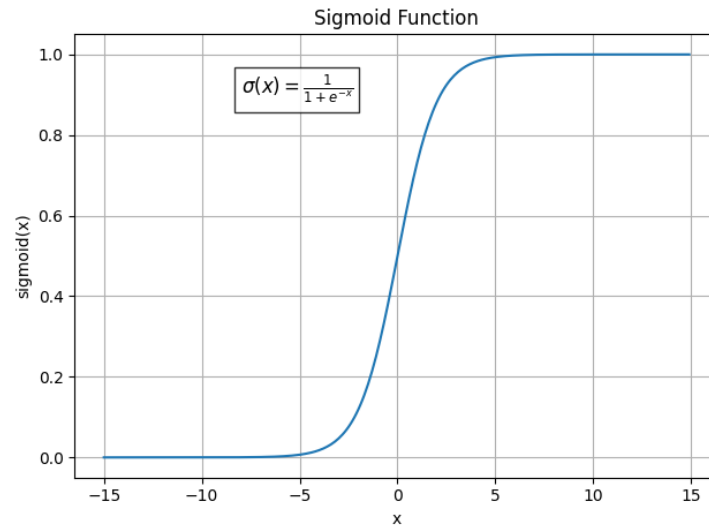
2. Áp dụng hàm Sigmoid lên z để tính xác suất:

$$P(y = 1|\mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}} \quad (2)$$

trong đó:

- $P(y = 1|\mathbf{x})$: Xác suất dự đoán nhãn là 1 (ví dụ: xác suất một người mắc bệnh).
- $\sigma(z)$: Hàm Sigmoid, ánh xạ z từ $-\infty$ đến $+\infty$ thành giá trị trong khoảng $[0, 1]$.

Tiếp tục ví dụ trên, nếu $z = 2$, thì $P(y = 1|\mathbf{x}) = \frac{1}{1+e^{-2}} \approx 0.88$, tức xác suất 88% người đó có nguy cơ bệnh.



Hình 1: Biểu đồ hàm Sigmoid với đầu ra nằm trong khoảng $[0, 1]$.

3. Quyết định nhãn dự đoán:

$$\hat{y} = \begin{cases} 1 & \text{nếu } P(y = 1|\mathbf{x}) \geq 0.5 \\ 0 & \text{nếu } P(y = 1|\mathbf{x}) < 0.5 \end{cases}$$

Ngưỡng 0.5 thường được sử dụng để phân loại, nhưng trong thực tế, ngưỡng này có thể thay đổi tùy bài toán. Chẳng hạn, trong chẩn đoán y khoa, ngưỡng có thể được đặt cao hơn (ví dụ: 0.7) để giảm nguy cơ bỏ sót trường hợp dương tính.

Hàm mất mát của Hồi quy Logistic

Để huấn luyện mô hình, ta cần một hàm mất mát (loss function) để đánh giá sự sai lệch giữa giá trị dự đoán và giá trị thực tế. Khác với hồi quy tuyến tính, hồi quy Logistic không thể sử dụng hàm Mean Squared Error (MSE) vì giá trị dự đoán không phải là một giá trị liên tục. Thực tế, giá trị dự đoán của hồi quy Logistic là một xác suất nằm trong khoảng $[0, 1]$, do đó ta cần sử dụng một hàm mất mát phù hợp. Dựa trên nguyên lý ước lượng hợp lý cực đại (Maximum Likelihood Estimation - MLE), ta xây dựng được hàm mất mát Cross Entropy:

$$\text{Loss} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (3)$$

Trong đó:

- N : Số lượng mẫu trong tập huấn luyện.
- y_i : Nhãn thực tế của mẫu thứ i .
- $\hat{y}_i$: Xác suất dự đoán của mẫu thứ i .

Tham khảo thêm tại:

- [Scikit-learn: Logistic Regression](#) - Tài liệu về hồi quy Logistic trong scikit-learn.

- [Machine Learning Cơ Bản: Logistic Regression](#) - Bài viết về hồi quy Logistic trên blog Machine Learning Cơ Bản.
- Christopher M. Bishop, *Pattern Recognition and Machine Learning*, Chapter 4.

Bài tập

Bài 1: Xây dựng Loss function của hồi quy Logistic

1.1. Entropy

Trong lý thuyết thông tin, *Entropy* đo mức độ bất định (uncertainty) của một phân phối xác suất. Với một biến ngẫu nhiên rời rạc X có phân phối xác suất $p_i = P(X = x_i)$, Entropy được định nghĩa như sau:

$$H(X) = - \sum_i p_i \log(p_i) \quad (4)$$

- Nếu một sự kiện gần như chắc chắn xảy ra ($p \approx 1$), Entropy thấp - ta đã biết trước kết quả.
- Nếu các sự kiện xảy ra đồng đều ($p \approx 0.5$ với bài toán nhị phân), Entropy cao - kết quả khó đoán nhất.

Ví dụ: Với bài toán nhị phân $y \in \{0, 1\}$ và $P(y = 1) = p$:

$$H(p) = -p \log(p) - (1 - p) \log(1 - p) \quad (5)$$

Entropy đạt giá trị lớn nhất khi $p = 0.5$ (hoàn toàn không chắc chắn) và bằng 0 khi $p = 0$ hoặc $p = 1$ (hoàn toàn chắc chắn). Đây là nền tảng để xây dựng hàm mất mát Cross Entropy ở phần 1.3.

1.2. Ước lượng hợp lý cực đại

Cho tập dữ liệu huấn luyện $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$, ta muốn tìm bộ tham số $\mathbf{w}$ sao cho mô hình giải thích tốt nhất dữ liệu quan sát được. *MLE* thực hiện điều này bằng cách tối đa hóa xác suất xảy ra của toàn bộ tập dữ liệu:

$$L(\mathbf{w}) = \prod_{i=1}^N P(y_i | \mathbf{x}_i; \mathbf{w}) \quad (6)$$

Vì tích của nhiều số nhỏ dễ gây hiện tượng tràn số (numerical underflow), ta lấy logarithm để chuyển tích thành tổng:

$$\log L(\mathbf{w}) = \sum_{i=1}^N \log P(y_i | \mathbf{x}_i; \mathbf{w}) \quad (7)$$

Với hồi quy Logistic, xác suất của mỗi mẫu được tính như sau:

$$P(y_i | \mathbf{x}_i; \mathbf{w}) = \hat{y}_i^{y_i} \cdot (1 - \hat{y}_i)^{1-y_i} \quad (8)$$

trong đó $\hat{y}_i = \sigma(\mathbf{w}^T \mathbf{x}_i)$ là xác suất dự đoán của mô hình. Thay vào log-likelihood:

$$\log L(\mathbf{w}) = \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (9)$$

Tối đa hóa log-likelihood tương đương với tối thiểu hóa phần âm của nó - đây chính là hàm mất mát Binary Cross Entropy sẽ được xây dựng ở phần tiếp theo.

1.3. Xây dựng hàm Binary Cross Entropy

Từ kết quả của MLE, ta định nghĩa hàm mất mát *Binary Cross Entropy* bằng cách lấy âm log-likelihood và chia trung bình trên N mẫu:

$$\mathcal{L}(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (10)$$

- Khi $y_i = 1$: chỉ có $y_i \log(\hat{y}_i)$ có hiệu lực. Nếu $\hat{y}_i \approx 1$ (dự đoán đúng), mất mát nhỏ. Nếu $\hat{y}_i \approx 0$ (dự đoán sai), mất mát rất lớn.
- Khi $y_i = 0$: chỉ số hạng $(1 - y_i) \log(1 - \hat{y}_i)$ có hiệu lực. Nếu $\hat{y}_i \approx 0$ (dự đoán đúng), mất mát nhỏ. Nếu $\hat{y}_i \approx 1$ (dự đoán sai), mất mát rất lớn.

Bài tập: Cho tập dữ liệu sau:

Mẫu	Nhãn thực tế y_i	Xác suất dự đoán $\hat{y}_i$
1	1	0.9
2	0	0.2
3	1	0.4

Thực hiện tính tay $\mathcal{L}(\mathbf{w})$ cho tập dữ liệu trên và nhận xét: mẫu nào đóng góp mất mát lớn nhất? Tại sao?

Bài 2: Xây dựng class Logistic Regression

1. Xây dựng hàm Sigmoid

```
1 def sigmoid(z):
2     return 1 / (1 + np.exp(-z))
```

2. Xây dựng hàm Loss function

```
1 def binary_cross_entropy(y_true, y_pred):
2     loss = -np.mean(y_true * np.log(y_pred) + (1 - y_true) *
3         np.log(1 - y_pred))
4     return loss
```

3. Hoàn thiện class LogisticRegression

```

1  class LogisticRegression:
2      def __init__(self, lr=0.01, num_iter=1000):
3          self.lr = lr
4          self.num_iter = num_iter
5          self.weights = None
6          self.bias = None
7
8      def sigmoid(self, z):
9          return
10
11     def binary_cross_entropy(self, y_true, y_pred):
12         return
13
14     def predict(self, X):
15         return
16
17     def fit(self, X, y):
18         return

```

Bài 3. Các phương pháp đánh giá mô hình hồi quy Logistic

Để đánh giá hiệu suất của mô hình phân loại nhị phân, ta thường sử dụng các chỉ số đánh giá dựa trên **Confusion Matrix** như: **Accuracy**, **Precision**, **Recall** và **F1-score**.

3.1. Confusion Matrix

Confusion matrix là bảng mô tả kết quả của mô hình phân loại trên tập dữ liệu kiểm thử. Trong đó chứa thông tin về các giá trị dự đoán của mô hình so với giá trị thực tế. Mỗi hàng của confusion matrix tương ứng với một lớp(class) thực tế, còn mỗi cột tương ứng với một lớp dự đoán. Cấu trúc của confusion matrix như sau:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Trong đó, 4 giá trị cơ bản của confusion matrix bao gồm:

- **True Positive (TP)**: Số mẫu thực sự là 1 và được mô hình dự đoán đúng là 1.
- **False Negative (FN)**: Số mẫu thực sự là 1 nhưng bị mô hình dự đoán nhầm thành 0.
- **True Negative (TN)**: Số mẫu thực sự là 0 và được mô hình dự đoán đúng là 0.
- **False Positive (FP)**: Số mẫu thực sự là 0 nhưng bị mô hình dự đoán nhầm thành 1.

Ví dụ Giả sử ta đã xây dựng một mô hình phân loại chó và mèo dựa trên hình ảnh. Chó được đánh nhãn là 1 và mèo được đánh nhãn là 0. Sau khi huấn luyện và kiểm thử, ta thu được confusion matrix như sau:

Trong trường hợp này:

	Predicted Dog	Predicted Cat
Actual Dog	90	10
Actual Cat	5	95

- **True Positive (TP):** 90 (Có 90 con chó được mô hình dự đoán đúng là chó).
- **False Negative (FN):** 10 (Có 10 con chó bị mô hình dự đoán nhầm thành mèo).
- **True Negative (TN):** 95 (Có 95 con mèo được mô hình dự đoán đúng là mèo).
- **False Positive (FP):** 5 (Có 5 con mèo bị mô hình dự đoán nhầm thành chó).

3.2. Công thức tính các chỉ số đánh giá

Accuracy (Độ chính xác) Accuracy đo lường tổng thể mức độ chính xác của mô hình, được tính bằng tỷ lệ số mẫu được dự đoán đúng trên tổng số mẫu. Tuy nhiên, nếu dữ liệu bị mất cân bằng, accuracy có thể không phản ánh đúng chất lượng mô hình.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (11)$$

Precision (Độ tập trung) Để có đánh giá chi tiết hơn, ta sẽ sử dụng Precision và Recall. Precision phản ánh mức độ chính xác của các dự đoán thuộc lớp dương (Positive), tức là trong số các mẫu được dự đoán là 1, bao nhiêu mẫu thực sự là 1.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (12)$$

Recall (Độ nhạy) Recall đo lường khả năng nhận diện chính xác các mẫu dương, tức là trong số các mẫu thực sự dương, có bao nhiêu mẫu được mô hình dự đoán đúng.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (13)$$

F1-score Khi có sự đánh đổi giữa Precision và Recall, ta có thể sử dụng F1-Score, là trung bình điều hòa của hai chỉ số này. Trung bình điều hòa có đặc tính phạt nặng khi một trong hai chỉ số quá thấp, phù hợp hơn trung bình cộng trong trường hợp này. F1-Score đặc biệt hữu ích khi cần mô hình cân bằng giữa hai yếu tố, nhất là khi một trong hai chỉ số quá cao hoặc quá thấp.

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (14)$$

Bài 4: Xây dựng mô hình dự đoán nguy cơ mắc bệnh tiểu đường

Pima Indians Diabetes là một tập dữ liệu nổi tiếng trong lĩnh vực học máy, được thu thập từ Viện Đái tháo đường, Tiêu hóa và Bệnh thận Quốc gia Hoa Kỳ (NIDDK). Tập dữ liệu bao gồm 768 mẫu, được thu thập từ những người phụ nữ thuộc cộng đồng Pima Indian tại Arizona, với 8 đặc điểm như nồng độ glucose, huyết áp, chỉ số BMI, ... cùng nhãn phân loại nhị phân: không mắc bệnh tiểu đường (0) hoặc mắc bệnh (1).

Yêu cầu 1: Huấn luyện mô hình Logistic Regression

- Đọc dữ liệu từ file **diabetes.csv** và chia thành hai tập huấn luyện và kiểm thử theo tỷ lệ 80-20.
- Tiến hành tiền xử lý dữ liệu: chuẩn hóa dữ liệu, xử lý missing values, ...
- Sử dụng class **LogisticRegression** đã xây dựng ở Bài 1 để huấn luyện mô hình trên tập huấn luyện.

Yêu cầu 2: Đánh giá mô hình

- Dự đoán nhãn trên tập kiểm thử và tính các chỉ số đánh giá: Accuracy, Precision, Recall, F1-score.
- Vẽ confusion matrix để phân tích kết quả dự đoán.

Yêu cầu 3: So sánh với thư viện scikit-learn

- Sử dụng class *LogisticRegression* trong thư viện scikit-learn để huấn luyện mô hình.
- So sánh kết quả dự đoán và các chỉ số đánh giá với mô hình tự xây dựng (trình bày kết quả dưới dạng bảng).

Bài 5: Xây dựng mô hình đánh giá chất lượng rượu vang

Vào năm 2009, với mục đích nghiên cứu chất lượng rượu vang dựa trên các phân tích hóa lý, Paulo Cortez cùng các cộng sự đã thu thập dữ liệu từ rượu vang đỏ ở vùng Vinho Verde, Bồ Đào Nha. Tập dữ liệu chứa thông tin về 1599 mẫu rượu, mỗi mẫu được đặc trưng bởi 11 thuộc tính như nồng độ cồn, axit citric, độ pH, v.v., và được các chuyên gia chấm điểm chất lượng trên thang từ 0 đến 10. Chi tiết tại: [Wine Quality](#).

Yêu cầu 1: Huấn luyện mô hình Logistic Regression

- Đọc dữ liệu từ file **winequality-red.csv** và chia thành hai tập huấn luyện và kiểm thử theo tỷ lệ 80-20.
- Tiến hành tiền xử lý dữ liệu: chuẩn hóa dữ liệu, xử lý missing values, ...
- Để phù hợp với bài toán phân loại nhị phân, ta có thể chuyển đổi nhãn chất lượng thành hai lớp: "0 - chất lượng thấp" (số điểm ≤ 5) và "1 - chất lượng cao" (số điểm > 5).
- Sử dụng class *LogisticRegression* đã xây dựng ở Bài 1 để huấn luyện mô hình trên tập huấn luyện.

Yêu cầu 2: Đánh giá mô hình

Thực hiện tương tự như Bài 4, Yêu cầu 2